Transaction

- Set of operations to perform a certain task.

ACID Properties

- A → Atomicity
- C → Consistency
- I → Isolation
- D → Durability

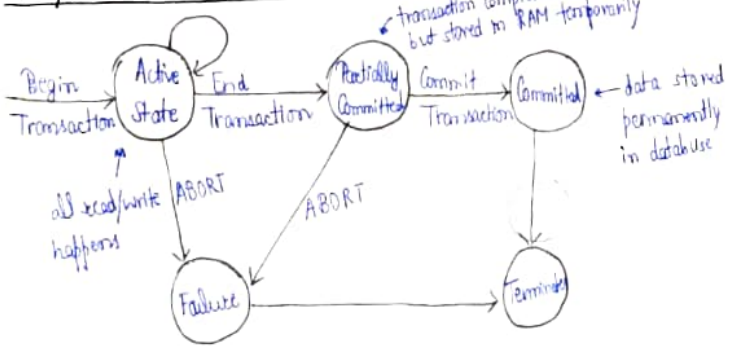
- Atomicity: either all steps of transaction is performed or none at all

(i) read(A)
(ii) A := A + 100
(iii) Write(A)
(iv) read(B)
(v) B := B + 100
(vi) write(B)

} either all 6 steps occur or none at all

- Consistency: each transaction takes the database from one consistent state to another consistent state
- Isolation: if multiple transactions are running simultaneously, all transactions should be isolated from all others
- Durability: after a transaction, the consistent state of the database must persist for some time

Lifecycle of Transaction



Types of Transaction:

(i) Serial schedule — one transaction runs after another
(ii) Concurrent schedule — more than one transaction runs simultaneously.

- In serial schedule, if I/O operation occurs, CPU goes idle

- Advantages of Concurrent Schedule:
  (i) increases system throughput
  (ii) reduces average response time
- Disadvantages of Concurrent schedule:
  (i) Lost update

T_1	T_2
read (A)	
A := A − 100	
	read (A)
	R := A * 0.2
	A := A + R
Write (A)	
read (B)	
	Write (A)
B := B + 100	
Write (B)	

here, this update is lost

(ii) Dirty read

T_1	T_2
read (A)	
A := A − 100	
write (A)	
	read (A) ← here, this is a dirty read
	A := A + 200
	write (A)
read (B)	
T_1 fails	

(iii) Unpredictable read

T_1	T_2
read (A)	
	read (A)
	A := A + 100
	write (A)
read (A)	

Same variable being read at different values at the same time

Scheduling

i) Serial Scheduling — after one transaction ends, another transaction will start

ii) Non-Serial Scheduling

- If we have $T_1, T_2, \ldots, T_n$ transactions running parallely and gives output like serial schedule, they're called serializability.

Serializable

- Conflict Serializability
- View Serializability

- We draw "precedence graph" to determine whether it is equivalent to serial schedule
- If graph doesn't gives a cycle, it is "conflict serializable".

Conflict Serializability

(1) If T_j → read operation (read(A))
after
T_i → write operation (write(A))



(2) If T_j → write operation (write(A))
after
T_i → read operation (read(A))

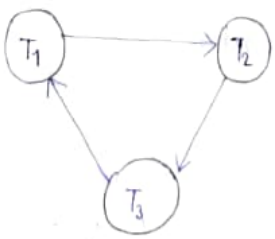


(3) If T_j → write operation (write(A))
after
T_i → write operation (write(A))



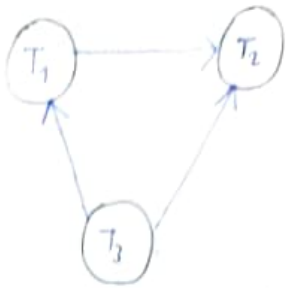
e.g -

T_1	T_2	T_3
	read(Z)	
	read(Y)	
	write(Y)	
		read(Y)
		read(Z)
read(X)		
write(X)		
		write(Y)
		write(Z)
read(Y)	read(X)	
write(Y)		
	write(X)	



∴ It forms a cycle, it is NOT equivalent to serial.

T_1	T_2	T_3
		read(Y)
		read(Z)
read(X)		
write(X)		
		write(Y)
		write(Z)
	read(Z)	
read(Y)		
write(Y)		
	read(X)	
	write(X)	
	read(Y)	
	write(Y)	



∵ It doesn't form a cycle

∴ It is equivalent to serial schedule, hence H is "conflict serializable".

T_1 : $R_1(P)$; $R_1(R)$; $W_1(P)$

T_2 : $R_2(R)$; $R_2(Q)$; $W_2(Q)$; $W_2(P)$

T_3 : $R_3(P)$; $R_3(Q)$, $W_3(Q)$

S_1 : $R_1(P)$, $R_2(R)$, $R_1(R)$, $R_1(P)$, $R_3(Q)$, $W_1(P)$, $W_3(Q)$, $R_2(Q)$, $W_2(Q)$, $W_2(P)$

T_1	T_2	T_3
read(P)		
	read(R)	
read(R)		
read(P)		
		read(Q)
write(P)		
		write(Q)
	read(Q)	
	write(Q)	
	write(P)	

View Serializable

- Two schedules S and S' are said to be view serializable if they are satisfying following conditions:

i) same set of transactions participate in both S and S'

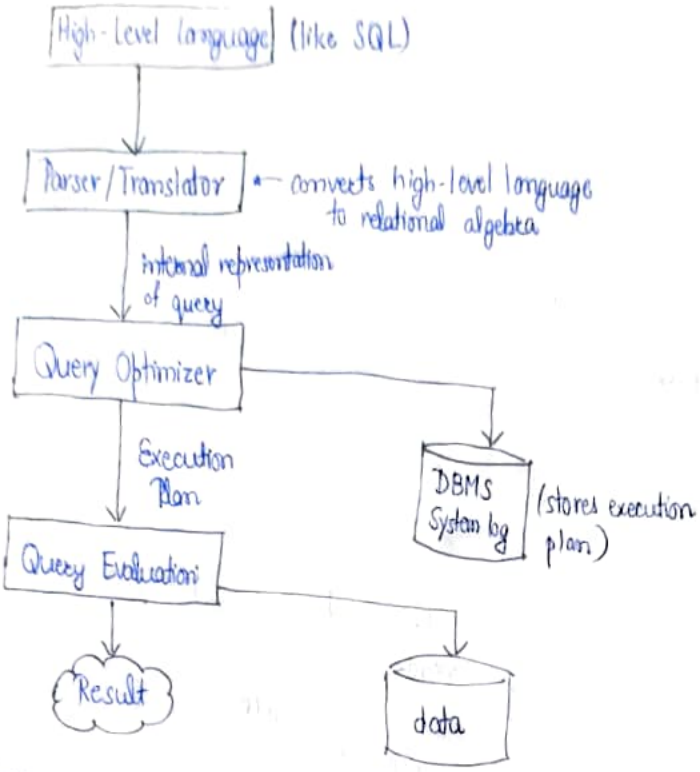
ii) if the transaction T_i reads initial value of a data item Q in schedule S, then T_i must read the same value of data Q in schedule S'.

iii) for each data item Q, if T_i executes read(Q) operation after write(Q) operation of T_j in schedule S, then T_i must execute read(Q) operation after write(Q) operation of T_j in schedule S'.

iv) if T_i performs final write operation in any data item Q in schedule S, then it must perform final write operation on same data item Q in schedule S'

Query Optimization

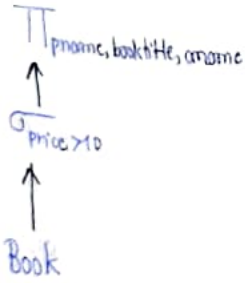
Query Processing:



Query Tree:

e.g - $\Pi_{pname, booktitle, aname}\left(\sigma_{price>10}(Book)\right)$

Tree:



(created before passing to optimizer)

- Query Optimization
  - Cost-based → we need to create query trees for each representation of query and then compute cost for each (large computational overhead)
  - Heuristic-based